POS Integration Roadmap — Business IQ Enterprise

> **Status:** Framework & Architecture Planning **Philosophy:** Build the integration layer so any new client's POS system can plug in with minimal config. Not perfect yet — we'll iterate.

Executive Summary

Business IQ Enterprise already has a complete inventory and sales schema (products, inventory, orders, orderItems, invoices, payments, customers, warehouses, inventoryTransactions). The POS integration doesn't rebuild this — it **bridges external POS systems into our existing data model** via a webhook-driven ingestion layer with vendor-specific adapters.

What We Already Have (Reuse — Don't Rebuild)

Domain	Existing Tables	Status
Products	`products` (SKU, barcode, price, costPrice, unit, minStockLevel, reorderPoint)	☑ Ready
Inventory	`inventory` (per product × warehouse), `inventory_transactions` (audit trail)	☑ Ready
Orders/Sales	`orders` (orderNumber, paymentMethod, paymentReference, paymentStatus), `order_items`	☑ Ready
Invoices	`invoices` (with KRA eTIMS fields: kraVerified, kraInvoiceNumber)	☑ Ready
Payments	`payments` (method, reference)	☑ Ready
Customers	`customers` (name, email, phone, balance, creditLimit)	☑ Ready
Warehouses	`warehouses` (code, isDefault — maps to POS locations/branches)	☑ Ready
Users	`users` (role hierarchy, permissions includes "pos", assignedWarehouses)	☑ Ready
Approval Flows	`approval_workflows`, `approval_requests`, `approval_decisions`	☑ Ready
Notifications	`notifications` table + scheduler agent	☑ Ready
Reports	`report-generator` agent + `saved_reports` table + PDF/Excel/Word/PPTX export	☑ Ready

Domain	Existing Tables	Status
Analytics	`insights-analyzer` agent (Python sandbox, statistical analysis)	☑ Ready

What We Need to Build

Component	Purpose
POS webhook routes	Receive sale/return/sync events from external POS systems
POS adapter layer	Vendor-specific payload normalization (M-Pesa, iKhokha, iTax, generic REST)
Sale ingestion service	Validate → deduplicate → create order + deduct inventory → check thresholds
Returns service	Validate original sale → reverse inventory → record return
Low-stock alert agent	Post-sale threshold check + daily sweep + alert dispatch
POS sync routes	Stock query API + catalog push for bidirectional sync
POS transactions table	Idempotency tracking for external transaction IDs
Webhook sources config	Per-POS-vendor auth, signature verification, field mapping

Architecture Principles

1. Single-Tenant, Not Multi-Tenant

Our architecture is **one codebase, one deployment per client**. There is no `client_id` filtering — the entire database belongs to one client. POS adapters don't need tenant isolation; they need **vendor isolation** (different POS systems sending different payloads to the same client's deployment).

2. Postgres First, Not KV

The original architecture guide uses KV for everything. We use **Drizzle + Postgres** for all transactional data. KV is only for caching, rate limiting, and idempotency dedup. Reasons:

- Relational joins (orders → order_items → products → inventory) are natural in Postgres
- Transactions with rollback (sale fails mid-inventory-deduct → rollback entire order)
- Existing schema already models all the entities
- Reporting queries need SQL, not KV scans

3. Routes + Services, Not Agent-Per-Operation

The guide proposes 10+ agents. In our architecture, most of this is **route + service layer** work:

- **Routes** (`src/api/pos.ts`) handle HTTP webhook endpoints
- **Services** (`src/services/pos-*.ts`) contain the business logic

- **Agents** are reserved for things that need AI or complex orchestration (low-stock analysis, anomaly detection, report generation — which we already have)

4. Adapter Pattern for Vendor Agnosticism

Each POS vendor gets an adapter module that normalizes their payload to our internal `PosTransaction` type. The core ingestion logic never sees vendor-specific fields.

```
M-Pesa Daraja  →  adapter-mpesa.ts     →  pos-ingestion.ts  →  orders + inventory
iKhokha        →  adapter-ikhokha.ts   →  pos-ingestion.ts  →  orders + inventory
iTax POS       →  adapter-itax.ts      →  pos-ingestion.ts  →  orders + inventory
Generic REST   →  (no adapter needed)  →  pos-ingestion.ts  →  orders + inventory
```

Phase 1 — Core POS Ingestion (Foundation)

1.1 Schema: `pos_transactions` Table

Idempotency and audit trail for external POS events. This is **not** a replacement for `orders` — it's the raw inbound event log that links to the order created from it.

```
pos_transactions (
  id              uuid PK,
  pos_vendor      varchar(50)    -- "mpesa", "ikhokha", "itax", "generic"
  pos_tx_id       varchar(255)   -- External transaction ID (idempotency key)
  pos_payload     jsonb          -- Raw payload as received
  status          varchar(30)    -- "received", "processed", "duplicate",
"failed", "returned"
  order_id        uuid FK → orders.id  -- The order created from this event
  warehouse_id    uuid FK → warehouses.id  -- Which branch/location
  error_message   text,
  processed_at    timestamptz,
  metadata        jsonb,
  created_at      timestamptz,
  updated_at      timestamptz,

  UNIQUE(pos_vendor, pos_tx_id)  -- Idempotency constraint
)
```

1.2 Schema: `pos_vendor_configs` Table

Per-vendor webhook configuration. Each deployment can have multiple POS vendors feeding data.

```
pos_vendor_configs (
  id              uuid PK,
  vendor          varchar(50)    -- "mpesa", "ikhokha", "itax", "generic",
"custom"
```

```
  display_name    varchar(255),
  is_active       boolean DEFAULT true,
  auth_type       varchar(30)    -- "hmac", "bearer", "basic", "none"
  auth_secret     text,          -- HMAC key, bearer token, or basic auth password
(encrypted)
  field_mapping   jsonb,         -- Maps vendor fields → our PosTransaction fields
  webhook_url     text,          -- Outbound webhook for bidirectional sync (push
catalog back)
  settings        jsonb,         -- Vendor-specific settings (e.g., M-Pesa
shortcode, till number)
  metadata        jsonb,
  created_at      timestamptz,
  updated_at      timestamptz
)
```

1.3 Service: `pos-ingestion.ts`

Core ingestion pipeline. Pure business logic, no AI.

```
Receive payload
  → Identify vendor (from route param or payload structure)
  → Adapter: normalize to PosTransaction
  → Idempotency check: SELECT FROM pos_transactions WHERE (vendor, pos_tx_id)
  → If duplicate: return { status: "duplicate", skipped: true }
  → DB Transaction:
      1. INSERT pos_transactions (status: "received")
      2. INSERT order + order_items (from normalized line items)
      3. For each line item: UPDATE inventory (decrement), INSERT
inventory_transaction
      4. UPDATE pos_transactions (status: "processed", order_id)
  → Post-processing (non-fatal):
      - Check low-stock thresholds for affected products
      - Queue notifications if any SKU breached reorder point
  → Return { status: "ok", orderId, itemsProcessed }
```

1.4 Service: `pos-returns.ts`

Return/refund processing.

```
Receive return payload
  → Find original pos_transaction by pos_tx_id
  → Validate: original exists AND status = "processed"
  → DB Transaction:
      1. INSERT pos_transactions (status: "returned", references original)
      2. INSERT order with negative amounts (or update original order status)
      3. For each returned item: UPDATE inventory (increment), INSERT
inventory_transaction (type: "return")
      4. UPDATE original pos_transaction (status: "returned")
  → Return { status: "ok", returnId }
```

1.5 Routes: `src/api/pos.ts`

```
// Webhook endpoints for external POS systems
POST /api/pos/ingest/:vendor        // Single sale event
POST /api/pos/ingest/:vendor/batch  // Batch sync (offline reconnect)
POST /api/pos/return/:vendor        // Return/refund
GET  /api/pos/stock                 // Query current stock (for POS catalog sync)
GET  /api/pos/catalog               // Full product catalog (for POS product sync)
POST /api/pos/catalog/push          // Push catalog updates to POS vendor webhook
```

Authentication: Each route validates the incoming request against `pos_vendor_configs.auth_secret` using the configured `auth_type` (HMAC signature, bearer token, or basic auth).

Phase 2 — Vendor Adapters

2.1 Adapter Interface

```ts
// src/services/pos-adapters/types.ts

interface PosTransaction {
  posVendor: string;
  posTxId: string;            // External transaction ID
  timestamp: Date;            // When the sale occurred at the POS
  items: PosLineItem[];
  subtotal: number;
  taxAmount: number;
  totalAmount: number;
  paymentMethod: string;      // "cash", "card", "mpesa", "bank_transfer"
  paymentReference?: string; // M-Pesa receipt, PDQ code, etc.
  customerId?: string;        // Phone number, loyalty ID, etc.
  customerName?: string;
  cashierId?: string;         // Staff/cashier identifier from POS
  locationId?: string;        // Maps to warehouse code
  currency?: string;
  rawPayload: unknown;        // Original payload for debugging
}

interface PosLineItem {
  sku?: string;
  barcode?: string;
  name: string;
  quantity: number;
  unitPrice: number;
  taxRate?: number;
  taxAmount?: number;
  discountAmount?: number;
  totalAmount: number;
```

```
}

interface PosAdapter {
  vendor: string;
  normalize(payload: unknown): PosTransaction;
  verifySignature?(payload: unknown, signature: string, secret: string): boolean;
}
```

2.2 M-Pesa Daraja Adapter

Receives C2B confirmation callbacks from Safaricom Daraja API.

```
// src/services/pos-adapters/adapter-mpesa.ts

// Maps Daraja C2B confirmation fields:
//   TransAmount     → totalAmount
//   BillRefNumber   → paymentReference (also used to look up product/SKU)
//   MSISDN          → customerPhone
//   TransID         → posTxId
//   TransTime       → timestamp (format: YYYYMMDDHHmmss)
//   BusinessShortCode → locationId (maps to warehouse by till number)
```

Note: M-Pesa C2B callbacks are payment confirmations, not itemized sales. The BillRefNumber often carries the account/reference — clients may encode SKU or order number here. The adapter should be configurable per client to handle their specific BillRefNumber format.

2.3 iKhokha Adapter

iKhokha is a South African POS terminal popular in East Africa. Their webhook format includes itemized line items.

```
// src/services/pos-adapters/adapter-ikhokha.ts
// Field mapping stored in pos_vendor_configs.field_mapping
```

2.4 iTax POS Adapter

KRA-compliant POS systems that transmit to eTIMS. Our webhook sits **alongside** the POS-to-KRA flow, never intercepting it.

```
// src/services/pos-adapters/adapter-itax.ts
// Receives a copy of the eTIMS-submitted invoice data
// Maps KRA invoice fields to our PosTransaction format
```

2.5 Generic REST Adapter

For any POS that can send a JSON webhook in a semi-standard format. Uses `pos_vendor_configs.field_mapping` JSONB to dynamically map fields.

```
// src/services/pos-adapters/adapter-generic.ts
// Reads field_mapping from vendor config:
// { "txId": "transaction.id", "amount": "transaction.total", "items":
"line_items", ... }
// Uses lodash _.get() style path resolution
```

Phase 3 — Low-Stock Alerts & Notifications

3.1 Post-Sale Threshold Check

After every sale ingestion, check if any affected product's stock dropped below its `reorderPoint`. This runs as a service function (not an agent) — deterministic business logic.

```
For each product in the sale:
  → Read inventory.quantity for (productId, warehouseId)
  → Read products.reorderPoint
  → If quantity <= reorderPoint:
      → Check recent notifications to avoid spam (dedupe by productId + 24h
window)
      → If no recent alert: INSERT notification for relevant users
```

3.2 Daily Sweep (Scheduler Agent — Already Exists)

The existing `scheduler` agent can run a daily cron task that sweeps all products across all warehouses for low-stock conditions. This catches any missed alerts from failed post-sale checks.

3.3 Alert Channels

- **In-app notifications** — Already built (`notifications` table + UI)
- **Email** — Via existing notification system
- **SMS (Africa's Talking)** — New integration for Kenyan market
  - Simple HTTP POST to `https://api.africastalking.com/version1/messaging`
  - Credentials via env vars: `AT_API_KEY`, `AT_USERNAME`, `AT_SENDER_ID`
  - Cheaper and more reliable than Twilio for Kenya/East Africa

---

Phase 4 — Batch Sync & Offline Reconnect

4.1 Batch Ingestion Endpoint

For POS systems that go offline and replay transactions on reconnect:

```
POST /api/pos/ingest/:vendor/batch
Body: { events: PosPayload[] }

→ Sort events by timestamp (chronological order)
→ Process each through the same ingestion pipeline
→ Idempotency via (pos_vendor, pos_tx_id) unique constraint handles replays
→ Return { processed: N, duplicates: N, errors: [...] }
```

4.2 Conflict Resolution

When an offline POS replays events:

- **Stock already adjusted manually?** — The inventory transaction audit trail shows both the manual adjustment and the POS sale. Reports can reconcile.
- **Product deleted/deactivated?** — Ingestion logs a warning in `pos_transactions.error_message` but doesn't fail. The line item is recorded with `productId: null` for manual review.

Phase 5 — Bidirectional Sync

5.1 Stock Query API

External POS systems query our stock levels for display:

```
GET /api/pos/stock?sku=ABC-123&warehouse=main
GET /api/pos/stock?warehouse=main  (all products for a location)
```

Returns current quantity, reserved quantity, reorder point, price, unit.

5.2 Catalog Push

When products are added/updated in Business IQ, push the changes to the POS system's webhook:

```
POST /api/pos/catalog/push
→ Read pos_vendor_configs.webhook_url for each active vendor
→ Format catalog data according to vendor's expected schema (reverse of adapter)
→ POST to vendor's webhook
→ Log result in pos_transactions (type: "catalog_push")
```

5.3 Product Sync on Change

Hook into the existing product CRUD routes. When a product is created, updated, or price changes:

- Queue a catalog push event
- The scheduler agent can batch-process push events every 5 minutes (avoids flooding the POS system)

Phase 6 — eTIMS Compliance (Kenya)

6.1 Parallel, Not Intercepting

Our system operates **alongside** the POS-to-KRA eTIMS flow:

```
Customer pays at POS
  ├── POS → KRA eTIMS (direct, we don't touch this)
  └── POS → Our webhook (parallel copy of the sale data)
```

We **never** sit between the POS and KRA. This is critical — delaying or interfering with eTIMS transmissions is a compliance violation.

6.2 eTIMS Fields in Our Schema

The `invoices` table already has KRA fields:

- `kra_verified` (boolean)
- `kra_verified_at` (timestamp)
- `kra_invoice_number` (varchar)

When the iTax adapter receives a sale event, it can populate these fields if the KRA invoice number is included in the POS payload.

Phase 7 — POS Dashboard & Analytics

7.1 Reuse Existing Infrastructure

The existing `data-science` agent, `insights-analyzer`, and `report-generator` already handle:

- Sales trends and forecasting
- Anomaly detection (z-scores on daily sales)
- Report generation (PDF, Excel, Word, PowerPoint)
- Custom SQL queries against the database

POS data flows into the same `orders`, `order_items`, and `inventory_transactions` tables — all existing analytics automatically include POS sales. No separate reporting pipeline needed.

7.2 POS-Specific Analytics

New analytical capabilities to add over time:

- **Cashier performance** — Sales volume by `pos_transactions.cashierId`
- **Peak hours** — Sales distribution by hour of day
- **Payment method breakdown** — Cash vs card vs M-Pesa split
- **Offline gap analysis** — Time gaps in `pos_transactions.timestamp` indicating connectivity issues
- **Reconciliation** — Match POS totals against bank/M-Pesa settlements

---

File Structure

```
src/
  api/
    pos.ts                          ← Webhook routes (ingest, batch, return,
stock, catalog)
  services/
    pos-ingestion.ts                ← Core ingestion pipeline (validate → dedup →
order → inventory)
    pos-returns.ts                  ← Return/refund processing
    pos-stock.ts                    ← Stock query + catalog push logic
    pos-alerts.ts                   ← Low-stock threshold checks + notification
dispatch
    pos-adapters/
      types.ts                      ← PosTransaction, PosLineItem, PosAdapter
interfaces
      adapter-mpesa.ts              ← Safaricom Daraja C2B normalization
      adapter-ikhokha.ts            ← iKhokha POS normalization
      adapter-itax.ts               ← KRA eTIMS-compliant POS normalization
      adapter-generic.ts            ← Dynamic field mapping from vendor config
      index.ts                      ← Adapter registry: getAdapter(vendor) →
PosAdapter
  db/
    schema.ts                       ← Add pos_transactions + pos_vendor_configs
tables
```

Environment Variables (New — POS-Specific)

```
# — POS Integration —
POS_WEBHOOK_ENABLED=true              # Master switch for POS webhook endpoints
POS_BATCH_MAX_EVENTS=500              # Max events per batch sync request
POS_DEDUP_WINDOW_HOURS=72             # How far back to check for duplicate
transactions

# — Africa's Talking (SMS Alerts) —
AT_API_KEY=                           # Africa's Talking API key
AT_USERNAME=                          # Africa's Talking username (sandbox or
production)
AT_SENDER_ID=                         # SMS sender ID (e.g., "BusinessIQ")

# — M-Pesa Daraja (if using M-Pesa adapter) —
MPESA_SHORTCODE=                      # Business short code or till number
MPESA_PASSKEY=                        # Daraja API passkey for signature
verification
MPESA_CALLBACK_URL=                   # Our webhook URL registered with Safaricom

# — Low-Stock Alerts —
LOW_STOCK_ALERT_CHANNELS=notification,email  # Comma-separated: notification,
email, sms
```

```
LOW_STOCK_DEDUP_HOURS=24              # Min hours between repeated alerts for same
SKU
```

Implementation Priority

Priority	Component	Effort	Dependencies
P0	`pos_transactions` + `pos_vendor_configs` schema	Small	None
P0	`pos-ingestion.ts` service (core pipeline)	Medium	Schema
P0	`src/api/pos.ts` webhook routes	Medium	Ingestion service
P1	`adapter-generic.ts` (dynamic field mapping)	Small	Ingestion service
P1	`adapter-mpesa.ts` (Daraja C2B)	Small	Ingestion service
P1	`pos-returns.ts` service	Medium	Ingestion service
P1	Post-sale low-stock threshold check	Small	Ingestion service
P2	Batch sync endpoint	Medium	Ingestion service
P2	Stock query API	Small	None
P2	Africa's Talking SMS integration	Small	AT credentials
P3	`adapter-ikhokha.ts`	Small	Ingestion service
P3	`adapter-itax.ts` + eTIMS field mapping	Medium	Ingestion service
P3	Catalog push (bidirectional sync)	Medium	Vendor webhook configs
P3	POS-specific analytics queries	Medium	Data accumulation

Key Decisions to Make Along the Way

1. **M-Pesa BillRefNumber format** — Each client uses BillRefNumber differently (some encode SKU, some use order number, some just use phone number). The adapter needs to be configurable per deployment.

2. **Multi-location stock** — When a POS sale comes in, which warehouse does it deduct from? Options:

   - POS payload includes location ID → map to warehouse code
   - Default to the POS vendor config's assigned warehouse
   - Fall back to the deployment's default warehouse

3. **Payment reconciliation** — Should we match POS payment amounts against bank/M-Pesa settlement reports? If yes, that's a Phase 7+ feature requiring a settlement ingestion pipeline.

4. **Offline conflict resolution** — When offline POS replays 200 events, and stock was manually adjusted during the offline period, how aggressive should the reconciliation be? Options:

    - Trust POS (replay all, overwrite manual adjustments)
    - Trust manual (skip POS events that conflict)
    - Flag conflicts for manual review (recommended)

5. **Real-time vs batch** — Some POS systems can send events in real-time (webhook per sale). Others only sync at end-of-day. The architecture handles both, but the alert timing differs.

POS Roadmap v0.1 — Framework and architecture for client integration. To be refined per client requirements.